KW307 SDK User Guide

24 GHz human-presence radar over UART — how it works, and the call that changes each part of it

SDK 2.17.1 · module firmware **2.10.0** and newer · 2026-10-13

Start at **Quick start**: the smallest program that works. The chapters after it follow the signal from the antenna to the target list, one section per block of the diagram, each ending with the calls that change that block. The **UART protocol** chapter at the end gives the bytes, for a host that does not run the SDK.

1 Quick start
- 1.1 Build integration
- 1.2 UART port functions
- 1.3 Minimal example program
- 1.4 Frame contents

2 Output modes
- 2.1 Output mode setup
- 2.2 Switching modes while running
- 2.3 Custom output frames

3 Error handling
- 3.1 Write verify

4 Signal path
- 4.1 Static clutter rejection and Doppler filters
- 4.2 Background update control
- 4.3 Distance, angle and speed extraction
- 4.4 Detection threshold
- 4.5 Tracking algorithm
- 4.6 Line-of-sight filtering
- 4.7 Detect range and field of view
- 4.8 Presence flag

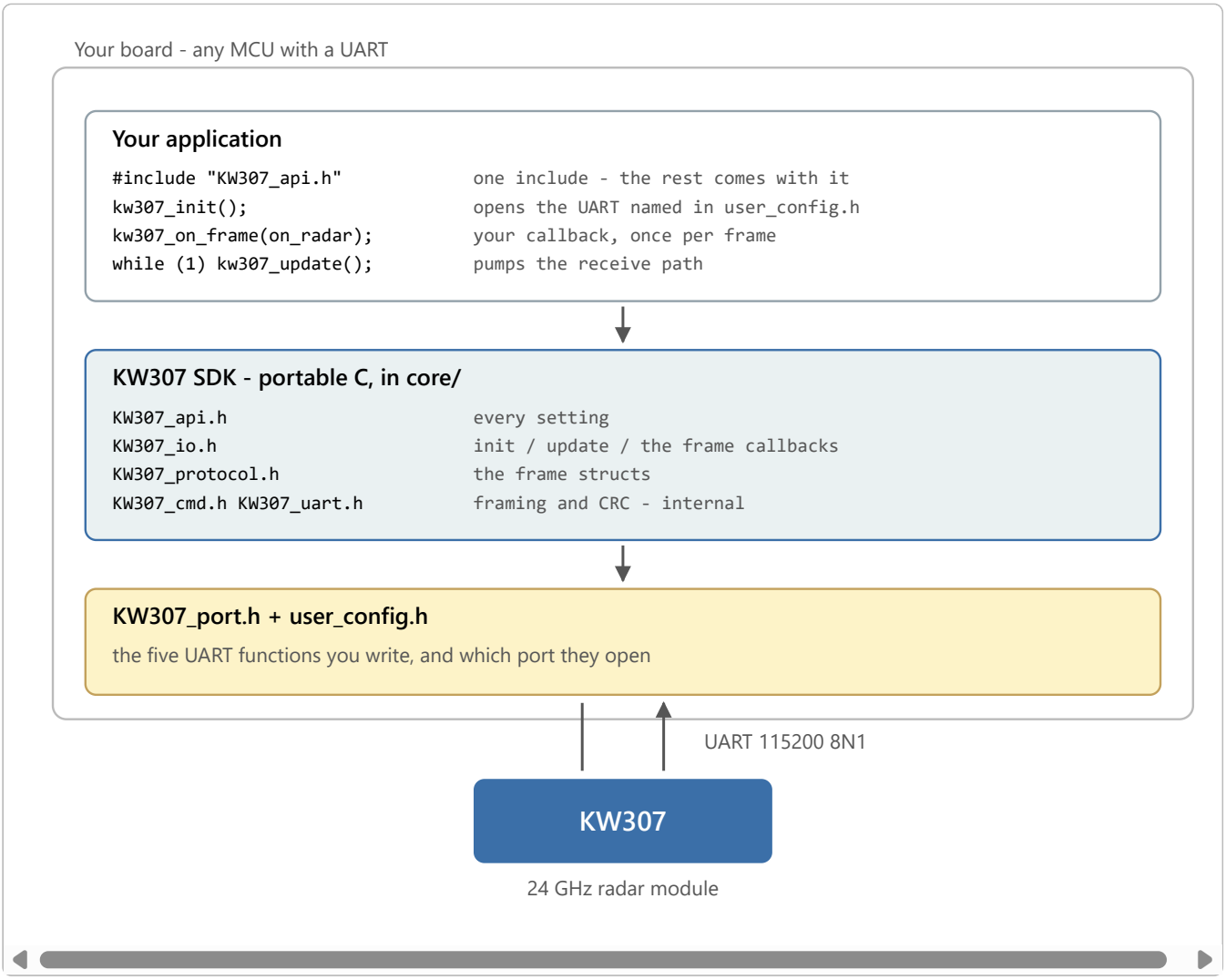
5 Application examples

6 Function reference
- 6.1 Session
- 6.2 Filters
- 6.3 Detection and reporting
- 6.4 Custom frames
- 6.5 Trackers
- 6.6 Front end and power

7 UART protocol
- 7.1 UART link settings
- 7.2 UART frame format
- 7.3 Write and read requests
- 7.4 Output frame layouts

1 Quick start

Settings take effect only once the module is running and reporting. Getting there takes five functions that you write, three calls that the SDK provides, and a main loop.



You write the top box and the bottom one. Everything between them ships as C source.

1.1 Build integration

Add every `.c` file under `core/` and every `.h` file beside them. There is no build system to adopt and nothing to configure at compile time. Add the files, add `core/` to your include path, and you have a C99 library with no dependency beyond `stdint.h` and `stdbool.h`.

1.2 UART port functions

The SDK never touches your hardware. It calls out to these, and you point them at whatever UART your board has:

```
int  kw307_port_open (const char* port_name, int baud_rate);
void kw307_port_close(void);
int  kw307_port_send (const uint8_t* buf, int len);
int  kw307_port_recv (uint8_t* buf, int max_len, int timeout_ms);
int  kw307_port_is_open(void);
```

Any host with a UART can run this. The SDK is plain C: no RTOS, no heap, no floating point, no vendor library. The five functions above are the complete port, whether the host is an ARM part, an AVR, bare metal or an RTOS.

`example_platform/port_template/` is a working skeleton with those five left as `TODO`. If your host is close to `STM32_HAL/`, `Arduino/`, `RPi_Pico/`, `pc_win32/` or `pc_linux/`, start from that one instead. Those are finished ports, not a list of what is supported.

The port they open is an ordinary UART at **115200 baud, 8 data bits, no parity, one stop bit, no flow control**, at **3.3 V TTL**. Those settings are in `user_config.h` next to the port you are writing, and the baud rate has to match the module's — 115200 is what it ships with.

> `kw307_port_recv()` must not block past `timeout_ms`, and must return what it has rather than waiting for a full buffer. Everything the SDK does on top — every read-back, every blocking command — is built on that one promise.

Every read-back waits about 200 ms (`KW307_READ_TIMEOUT_MS`) for the module's answer and then returns `KW307_ERR_TIMEOUT`, so no call can hang.

1.3 Minimal example program

```
#include "KW307_api.h"

static void on_radar(const KW307Frame_t* f)
{
    set_led(f->presence_flag);                  /* 1 while someone is there */
}

int main(void)
{
    board_init();                           /* your clocks, pins, UART */

    if (kw307_init() != KW307_OK) return 1;  /* opens the port, starts the link */
    kw307_on_frame(on_radar);                /* your callback, one per frame */

    for (;;) kw307_update();                 /* reads the UART, runs callbacks */
}
```

`kw307_update()` does not block. Call it as often as your loop runs; the module sends a frame about twenty times a second and `on_radar` fires once per frame.

That program never configures the module, and it works: the module boots with the settings in its own flash and starts streaming. Configuration is what the rest of this guide is about, and it needs no second header — `KW307_api.h` is the only one you ever include. It pulls in the rest, so every call in this guide is in scope from that one line.

> Call `kw307_check_fw_version()` once after `kw307_init()`. It returns `0` when the firmware is compatible, `1` on firmware 2.7.6, and a negative code if the module did not answer. On 2.7.6 the calls marked **Not available on firmware 2.7.6** return `KW307_ERR_NOT_SUPPORTED`.

1.4 Frame contents

```
typedef struct {
    uint16_t       frame_index;   /* frame counter */
    uint8_t        presence_flag;    /* 1 = presence this frame */
    KW307Tracker_t moving[2];     /* walking, gesturing */
    KW307Tracker_t stationary[1]; /* a seated person, micro-motion */
} KW307Frame_t;
```

The pointer is valid for the length of the callback. Copy out what you need.

> **Use a slot only when its `state` says Valid.** `state != 0` is not that test. The other non-zero values mean a target is being confirmed, or is deliberately hidden, and using those makes one person appear as two. Valid is `1` in the simple reporting form, the default. After `kw307_set_tracker_state_advanced_mode(true)`, a moving target is Valid at `3` instead. `core/KW307_protocol.h` lists every state value.

Each slot also carries `counter`, which rises while the target keeps being seen and falls when it is missed. It is useful as a confidence or fade level.

2 Output modes

The quick start showed one output. There are three, and the difference between them is only **how much each frame carries**. Every one of them reports presence, two moving targets and one stationary target — that part never changes. What changes is what is added on top.

MODE	WHAT IT ADDS	PAYLOAD	YOUR CALLBACK	STRUCT
`KW307_OUTPUT_NORMAL`	— the default frame	27 B	`kw307_on_frame()`	`KW307Frame_t`
`KW307_OUTPUT_NORMAL_WITH_SPD`	how fast each moving target is going	34 B	`kw307_on_frame_spd()`	`KW307FrameSpd_t`
`KW307_OUTPUT_NORMAL_WITH_SPD_RG`	the above, plus the four nearest stationary range gates	66 B	`kw307_on_frame_spd_rg()`	`KW307FrameSpdRg_t`

There is also `KW307_OUTPUT_OFF`, which stops the stream. The radar keeps running; frames are no longer sent.

If you are not sure: use `KW307_OUTPUT_NORMAL_WITH_SPD`. It is the default frame plus the one field most products end up wanting, and it costs 7 bytes a frame.

The three structs are in **`core/KW307_protocol.h`**, field by field: what each one means, what `state` is allowed to be and which value means a real target, what the speed units are, and what the range gates are and are not. Please refer to that file for each field's details.

2.1 Output mode setup

Whichever mode you pick, the two steps are the same — register the callback, then select the mode:

```
kw307_on_frame_spd(on_radar);                    /* 1. where frames go   */
kw307_set_output(KW307_OUTPUT_NORMAL_WITH_SPD);  /* 2. what to send      */

while (1) { kw307_update(); }                    /* 3. read the UART     */
```

Register before you select. Frames start arriving as soon as the module is told to send them. If the callback is not in place yet, the first few frames are lost.

`kw307_update()` reads the UART and calls your callback. If it is not called regularly, no frames are delivered, so check this first when no frames arrive.

2.2 Switching modes while running

You can change mode at any time — a calibration typically goes into `KW307_OUTPUT_NORMAL_WITH_SPD_RG`, takes its reading, and returns to `KW307_OUTPUT_NORMAL_WITH_SPD`:

```
kw307_on_frame_spd(on_normal);        /* register both, once, at start-up */
kw307_on_frame_spd_rg(on_gates);

...

kw307_set_output(KW307_OUTPUT_NORMAL_WITH_SPD_RG);   /* measure   */
...
kw307_set_output(KW307_OUTPUT_NORMAL_WITH_SPD);      /* and back  */
```

Register every callback you might use, once, at start-up. There is nothing to unregister when you switch: only the selected mode's frames are sent, so only its callback fires. Frames still in flight from the previous mode are matched to the right callback on the way in, not mis-read.

2.3 Custom output frames

`KW307_OUTPUT_NORMAL_WITH_SPD` and `KW307_OUTPUT_NORMAL_WITH_SPD_RG` are two fixed shapes of one frame, `0xF3`, which can carry any mix of eight blocks. When neither fits, pick the blocks yourself: each call below switches one block on or off in a `KW307OutputConfig_t`, and `kw307_set_output_config()` sends the result in one write.

CALL	BLOCK
`kw307_out_presence(c, on)`	presence flag
`kw307_out_moving_trk(c, on, full, speed)`	the two moving trackers; `full` = whole records rather than state only, `speed` = each one's speed
`kw307_out_stationary(c, nearest, second)`	the nearest and second-nearest stationary targets
`kw307_out_stationary_trk(c, on, full)`	the four stationary-band trackers, with their raw state: 0 Empty, 1 Checking, 2 Birth, 3 Valid
`kw307_out_room_max(c, on)`	the room's peak magnitude, per band
`kw307_out_rg(c, start_rg, end_rg, moving, stationary, mag_only)`	range gates `start_rg` ~ `end_rg`, per band

Start from `kw307_out_clear()`, or from the shape you are nearest to — `kw307_out_normal_with_spd()` and `kw307_out_normal_with_spd_rg()` fill in those two exactly. `KW307_OUTPUT_NORMAL_WITH_SPD_RG`, with gates 0~7 instead of

0~3:

```
KW307OutputConfig_t c;

kw307_out_normal_with_spd_rg(&c);                 /* SPD_RG shape ...     */
kw307_out_rg(&c, 0, 7, false, true, false);       /* ... gates 0~7        */

kw307_on_frame_f3(on_f3);                         /* every shape lands here */
kw307_set_output_config(&c);
```

```
static void on_f3(const KW307FrameF3_t* f)
{
    int g;
    if (f->blocks & KW307_F3_PRESENCE) { /* f->presence_flag is in this frame */ }
    for (g = f->start_rg; g <= f->end_rg; g++)
        printf("gate %d: mag %u\n", g, f->rg[g][KW307_BAND_STATIONARY].mag);
}
```

- **Every block is in the struct whether it was sent or not.** What the frame

did not carry is zero; `blocks` says which is which. Test the bit, not the value.

- **`rg[]` is indexed by the gate number**, so `rg[5]` is gate 5 whatever range

you chose.

- **`kw307_out_len(&c)` tells you the frame size before you send it** — 6 to

270 bytes. At 20 frames a second the largest wants 115200 baud, and fills the default 256-byte receive buffer in about 45 ms: call `kw307_update()` more often than that, or enlarge `KW307_UART_RX_BUFSIZE`.

- **The module forgets the shape at power-off** and comes back on the default

frame. Send it again after a reset.

`kw307_on_frame_f3()` sits alongside the two callbacks above rather than replacing them: a frame of either mode's exact shape still goes to `kw307_on_frame_spd()` / `kw307_on_frame_spd_rg()` if those are registered, and every other shape comes to `kw307_on_frame_f3()`. Writing your own transport? `kw307_dec_frame_f3()` decodes any shape from the payload bytes.

For tuning a board there is a ready-made shape: `kw307_set_output_rg(start_rg, end_rg, en_moving_trk, en_stationary_trk)` sends both bands' gates in full, with the moving and/or stationary trackers alongside so you can see what each gate saw and what the trackers made of it.

3 Error handling

Every call that sends to or reads from the module returns `int`. `KW307_OK` is zero and every failure is negative, so the ordinary test is one line:

```
if (kw307_set_detect_range(30, 400) != KW307_OK) { /* handle it */ }
```

That catches everything. Tell the four apart only where your response differs; the constants are in `KW307_io.h`.

	WHAT TO DO
The link — the UART is not there (`KW307_ERR_PORT_CLOSED` and the other `KW307_ERR_PORT_*` codes). `kw307_is_open()` says so without sending anything.	Stop configuring. `kw307_init()` is what brings it back.
The argument — a value outside the range the call accepts (`KW307_ERR_PARAM`).	Check the value against the call's range. Nothing was sent to the module.
The answer — nothing came back (`KW307_ERR_TIMEOUT`), or the module rejected the value (`KW307_ERR_NACK`).	After a timeout, a retry is reasonable, except for `kw307_save_settings()`, `kw307_factory_reset()` and `kw307_clear_state()`: it is not known whether the module acted. After a rejection, correct the value first; `kw307_last_nack_status()` holds the reason until the next rejection.
The firmware — the module runs firmware 2.7.6, which does not have this call (`KW307_ERR_NOT_SUPPORTED`).	Not a fault; do not retry. Leave the feature out on that module.

> **A timeout is not always the module.** A blocking call waits about 200 ms (`KW307_READ_TIMEOUT_MS`) for a reply that only arrives if something is draining the port. A stalled loop, a disabled interrupt, a buffer never emptied — any of them time out every read while the module is sending perfectly well.

> **A rejection is the module disagreeing with the value, not the link failing.** `kw307_last_nack_status()` gives the reason.

3.1 Write verify

A setter returns as soon as the command is on the wire, so `KW307_OK` means **sent**, not **applied**. To make every write wait for the module's ACK, call `kw307_set_write_verify()` once at start-up. A rejected value is then returned as `KW307_ERR_NACK`; a timeout is not returned, but is counted and passed to the callback you give it. `kw307_write_stats_get()` counts the writes

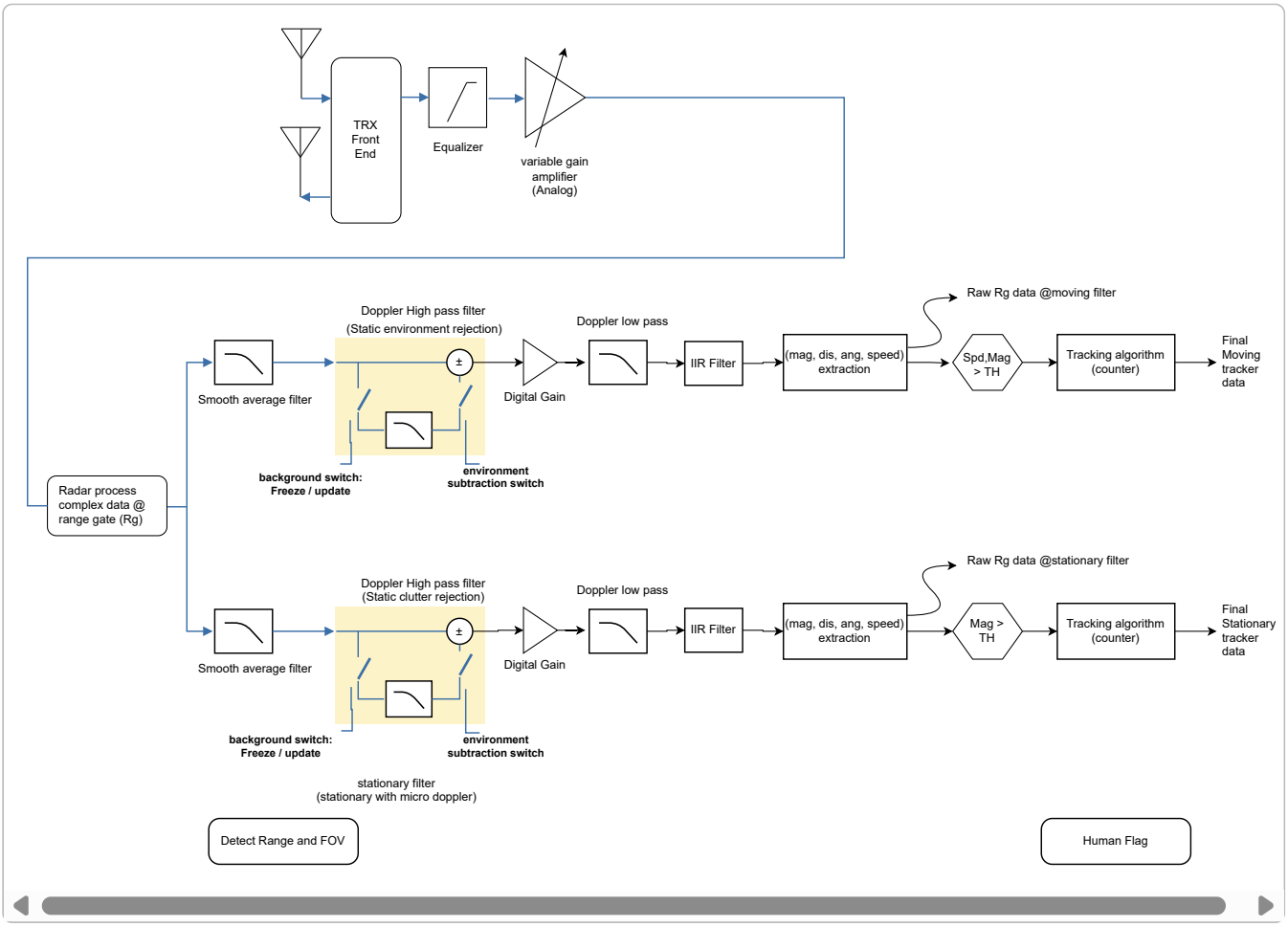
that were verified, rejected, timed out or sent unverified since the last `kw307_write_stats_reset()`, which tells an unreliable link apart from a rejected setting. Write verify is intended for configuration tools and production fixtures; leave it off in a product's main loop.

4 Signal path

In the KW307 radar system, motion objects are categorised into two types:

- **Moving objects** — macro-movements such as walking or running.
- **Stationary objects with micro-Doppler** — micro-movements such as breathing, heartbeat, or subtle body shifts.

The radar hardware processes incoming signals through two parallel bands: the **moving band** and the **stationary band**. Both bands share a similar architecture and the same configurable parameters.




```
The moving band takes 20 to 160 Hz, the stationary band 0.31 to 10 Hz.
```

Every value the module reports comes out of those two paths, and every setting in this guide changes one block in one of them. Left to right:

THE BLOCK	WHAT IT DOES	YOURS TO CHANGE
Analog front end	Transmit and receive, the equalizer that compensates the signal's loss over range, and the baseband gain applied to the signal before it is digitised.	The baseband gain, and the equalizer order.
Complex data per range gate	The radar's own output: one in-phase / quadrature pair per range gate.	Which gates are in play, via the detection zone.
Smooth average	A short average over frames that takes the noise off each gate's signal before the Doppler filters see it.	Nothing. It is fixed, and it is the reason a single noisy frame does not become a target.
Doppler high-pass	Removes the stationary clutter — floors, walls, furniture, the antenna's own cross-talk. Two switches inside it decide **how** it is removed.	The corner, and the two switches.
Digital gain	A gain step after the clutter is removed, one setting for both bands. The baseband gain has to carry clutter and target together; this one carries only the residual signal, so it lifts a weak target without saturating the front end.	The step, 1x to 8x.
Doppler low-pass	Removes Doppler faster than the band covers, which is what separates the two bands.	The corner.
Range-gate IIR	An IIR filter that averages each gate's complex data over frames, one coefficient per band.	The coefficient, 0~1000.
Extraction	Turns each gate's filtered signal into a magnitude, a distance, an angle and a speed.	Nothing.
Detection threshold	Decides which gates become candidates for the tracker — fast enough for the band, and strong enough. It only decides which gates can form a target; the reported values are unchanged.	The threshold, per band.
Tracking	Decides which candidates are the same target across frames, and when a target is confirmed enough to report.	The counters.

The **detection zone** and the **presence flag** sit beside the two bands rather than inside them: they apply to every target rather than to one point in the pipeline. The detection zone says which gates and which angles count at all; the presence flag is the presence answer on its own.

4.1 Static clutter rejection and Doppler filters

In radar signal processing, rejecting static clutter — floors, walls, stationary furniture and antenna TX-to-RX cross-talk — is essential to isolate target motion. Static environmental clutter appears at DC (0 Hz).

- Micro-Doppler signals, such as human respiration, typically start around **0.3 Hz**.
- Macro-movements, such as walking, start around **20 Hz**.

The static clutter rejection circuit is equivalent to a Doppler high-pass filter. Setting a lower high-pass corner frequency enables detection of micro-Doppler targets, but increases susceptibility to environmental vibration clutter.

A Doppler low-pass filter filters out fast movement. If you do not want to detect fast movement, setting the low-pass corner lower keeps only slower-moving objects and lowers the environment noise.

Sensitivity levels

One call per band selects that band's Doppler corners for you.

```
kw307_set_motion_sensitivity(level);       /* 0~9, or -1 for any motion */
kw307_set_stationary_sensitivity(level);   /* 0~9, stationary band      */
```

Ten levels per band, from `0` (most sensitive) to `9` (least sensitive); the moving band also accepts `-1`. Each level is a row of settings picked together as a set: where the band's filters sit, and how little movement still counts.

LEVEL	HIGH-PASS	MIN SPEED	CONFIRM FRAMES	SPEED CLIP	MIN SPEED SUM
-1	0.16 Hz (corner 7, nd 3)	1	4	2	0 (off)
0	10 Hz (corner 4)	250	4	500	1000
1	20 Hz (corner 3)	250	4	500	1000
2	20 Hz (corner 3)	250	8	500	2000
3	20 Hz (corner 3)	500	4	1000	2000
4	40 Hz (corner 2)	500	4	1000	2000
5	40 Hz (corner 2)	500	8	1000	4000
6	40 Hz (corner 2)	1000	4	2000	4000
7	80 Hz (corner 1)	500	8	1000	4000
8	80 Hz (corner 1)	1000	4	2000	4000
9	80 Hz (corner 1)	1000	8	2000	8000

Moving band. `-1` is one step below 0: the lowest corner the filter has, the lowest speed that counts as movement, and the added-up speed check (`min_speed_sum`) off entirely.

LEVEL	HIGH-PASS	LOW-PASS	RECEIVER GAIN
0	0.16 Hz (corner 7, nd 3)	10 Hz (corner 8)	0 dB
1	0.31 Hz (corner 7, nd 2)	10 Hz (corner 8)	0 dB
2	0.62 Hz (corner 7, nd 1)	10 Hz (corner 8)	0 dB
3	1.2 Hz (corner 7, nd 0)	10 Hz (corner 8)	0 dB
4	2.5 Hz (corner 6, nd 0)	10 Hz (corner 8)	0 dB
5	5 Hz (corner 5, nd 0)	10 Hz (corner 8)	0 dB
6	10 Hz (corner 4, nd 0)	20 Hz (corner 16)	0 dB
7	10 Hz (corner 4, nd 0)	20 Hz (corner 16)	-6 dB
8	20 Hz (corner 3, nd 0)	40 Hz (corner 32)	-6 dB
9	40 Hz (corner 2, nd 0)	80 Hz (corner 64)	-6 dB

Stationary band. A stationary level sets **both** corners — the high-pass and the low-pass — and 7~9 additionally take 6 dB out at the receiver. A level never changes the thresholds set by `kw307_set_mag_threshold()`. Levels 6 and 7 are the same filter and differ only in that 6 dB.

> **The receiver's gain is ahead of the band split.** Levels 7~9 quieten the moving band by the same 6 dB, not the stationary band alone. If that matters, stay at 6 or below and raise the stationary threshold yourself with `kw307_set_mag_threshold()`.

> **Coming from SDK 2.5 or earlier.** Motion sensitivity took `(level, byp_spd)`, and `level` was ignored whenever `byp_spd` was set. `byp_spd = 1` is now `level = -1`; every other level is unchanged. Both sensitivity calls take `int8_t` so `KW307_LEVEL_NONE` (the value a getter reports when the band matches no level) fits, and `kw307_get_motion_sensitivity()` no longer takes a `byp_spd` pointer.
>
> **Coming from SDK 2.9.0 or earlier.** Stationary levels 7~9 used to double both detection thresholds (350 / 100 to 700 / 200). To keep those thresholds, call `kw307_set_mag_threshold(700, 200)`.

Direct corner frequency settings

The same four filters a level sets. Use these when no level lands where you want it.

```
kw307_set_moving_hpf_corner(corner, nd);       /* moving     high-pass, corner 0~7, nd 0~3 */
kw307_set_moving_lpf_corner(corner);           /* moving     low-pass, 0~255              */
kw307_set_stationary_hpf_corner(corner, nd);   /* stationary high-pass                     */
kw307_set_stationary_lpf_corner(corner);       /* stationary low-pass                      */
```

At the default 2 kHz sampling rate:

```
high-pass   2000 / 2^corner / 2^(nd + 1) / 2π   Hz
low-pass    2000 * corner / 256 / 2π            Hz
```

The high-pass halves with every step of `corner`; the low-pass is linear in it.

> **The high-pass depends on `corner + nd` and on nothing else.** So `4/2` and `6/0` are the same 2.5 Hz. `nd` is the downsample factor, 0~3. Eight corners and four `nd` give eleven distinct corner frequencies, each half the one above it. Reach for `nd` only once `corner` is at 7 and the corner still has to come down.

CORNER + ND	0	1	2	3	4	5	6	7	8	9	10
high-pass, Hz	160	80	40	20	10	5	2.5	1.2	0.62	0.31	0.16

Defaults, and the two bands as one picture:

	HIGH-PASS	LOW-PASS	WHAT IS LEFT
Moving	20 Hz, `3 / 0`	160 Hz, `128`	walking, gesturing, a hand arriving
Stationary	0.31 Hz, `7 / 2`	10 Hz, `8`	breathing, a small shift in a chair

That is motion level 2 and stationary level 1. The moving band starts where macro movement starts, so between 10 and 20 Hz both bands are rolling off. To make the two bands meet, use motion level 0, which sets the moving high-pass corner to 10 Hz.

> Setting a corner directly takes the band off every level, so `kw307_get_motion_sensitivity()` will report `KW307_LEVEL_NONE` afterwards unless the values happen to land on one. This is expected, not an error.

> The low-pass also sets the band's anti-alias limit before decimation. Raising it well past the band's own range adds no detections. It lets faster movement fold back into the range the band does report, which reads as targets that are not there. We recommend keeping the default low-pass corners, which are the tested settings.

4.2 Background update control

An Update / Freeze background switch controls background adaptation in the static clutter rejection.

- **Update** (the default) — operates as a standard Doppler high-pass, continuously updating the static clutter estimate.
- **Freeze** — freezes background adaptation. This effectively extends the high-pass corner frequency down to DC (0 Hz), so a newly added stationary object is detected against the memorised background.

```
kw307_set_moving_background(update);           /* true = keep adapting */
kw307_set_stationary_background(update);
```

Environment subtraction switch

The high-pass is the signal minus a low-pass of itself. The switch above acts on that low-pass, the background estimate; this one acts on the subtraction.

- **On** (the default) — the estimate is subtracted, and the output is high-passed.
- **Off** — the signal passes with the environment still in it, which is the path for measuring the environment.

```
kw307_en_moving_env_subtraction(on);
kw307_en_stationary_env_subtraction(on);
```

Stationary result hold

```
kw307_en_stationary_hold(on);     /* true = hold the stationary result */
```

This holds the stationary tracker result, so a change of filter state does not reach what you read. Every stationary slot keeps the distance, angle, magnitude and state it had when the hold went on. The moving band is not held.

Use the hold while you change the environment subtraction switch. Changing that switch moves the output path: the signal steps, and the estimate behind it needs seconds to settle. The module cannot tell that swing from a real target arriving or leaving, and the hold keeps it out of the stationary result. The background switch does not need the hold — it changes what the filter learns, not where the signal goes.

> **Turn the hold off when you have finished.** The hold has no timeout. While it is on, the stationary output is frozen, so if the host turns it on and then stops communicating, the module keeps reporting the room as it was when the hold started.

Analog high-pass (DC cancellation)

The receiver also cancels DC with an analog high-pass filter, before the signal is split into the two bands, so these switches act on both bands. There are two switches, both on at power-up:

```
kw307_en_analog_hpf(on);          /* the analog high-pass        */
kw307_en_analog_hpf_swcap(on);    /* its switched-capacitor path */
```

Turning the analog high-pass off extends its corner down to DC. The switched-capacitor path is the same high-pass at a lower corner, so a target that never moves needs **both** off; either one left on removes it. Expect a noisier signal once they are off:

the offset that was being removed is now in the signal.

4.3 Distance, angle and speed extraction

This block turns each range gate's complex (I/Q) data into a magnitude, a distance, an angle and a speed.

Ahead of it, the range-gate IIR, one per band, averages each gate's complex data over frames:

```
kw307_set_RangeGate_IIR(moving, stationary);     /* 0~1000, default 150 (0.15) both */
```

The value is the weight given to each new measurement, in thousandths:

```
output  =  k/1000 x new measurement  +  (1 - k/1000) x previous output
```

So the default 150 counts a new reading as 0.15 and what it already had as 0.85. Higher follows the measurement more closely, lower holds the previous value harder, and 1000 is no averaging at all.

One coefficient covers all four quantities: magnitude, distance, angle and speed. The tracker has its own smoothing, in **Tracking algorithm** below.

4.4 Detection threshold

Between the readings and the tracker there is one more decision: which range gates become candidates. A gate has to be moving at the right speed for its band **and** be strong enough, or it takes no part in forming a target.

```
kw307_set_mag_threshold(moving, stationary);   /* the threshold, per band. Default 350 / 100 */
```

It is a minimum, not a target. The default suits most rooms. Raise it only when the room is noisier than the default allows for, and measure the room first with the noise floor report below.

Noise floor report

```
kw307_en_noise_floor_report(on);     /* off by default */
```

With it on, an **empty** target slot reports the room's peak magnitude in its `mag` instead of zero — the peak an empty room gives back, which is what a real target has to exceed.

- It reports only while **nobody is detected**. The moment the presence flag goes to 1 the slot reads 0 again: with a person in the room the peak magnitude **is** the person. It gates on **people**, not on objects, so anything else left in the room is part of the reading.

- Only the **first moving slot** and the **nearest stationary slot** carry it. The measurement is per band, not per slot, so the rest stay 0.
- It is the room's **peak**, not its average.

> Use `state` to tell whether a slot holds a target, never `mag`. With this on an empty slot carries a non-zero magnitude, and code that treats `mag != 0` as a target will report a target in an empty room.

4.5 Tracking algorithm

It is common for radar systems to use a tracking algorithm, such as a Kalman filter, to build trajectory information for detected objects. The main reason is that range-gate raw data often contains impulse environmental noise, and discrepancies between measurement and prediction. With a tracking algorithm the radar can filter out the impulse noise and provide a much smoother trajectory.

A tracker counter is used to maintain the stability and credibility of the trajectory. Each slot carries a count that rises while the target keeps being seen and falls when it is missed:

```
kw307_set_moving_tracker_cnt(cnt_limit, display_th);
kw307_set_stationary_tracker_cnt(cnt_limit, display_th);
```

- `cnt_limit` — the counter's upper limit, in frames: how long a target survives without being seen again. Defaults: 32 moving, 64 stationary.
- `display_th` — the count at which the target is reported. Default 8 for both, and it must not exceed `cnt_limit`.

Beyond the counters, each band sets what it takes to report a target, and how steadily the reported position follows each new measurement:

```
kw307_set_motion_threshold(confirm_frames, min_speed, speed_clip, min_speed_sum);
kw307_set_moving_smoothing(smoothing);
kw307_set_stationary_tracker_opt(confirm_frames, smoothing, match_distance_cm);
```

- `confirm_frames` (moving) — the number of frames a new moving target's speed is added up over before it is reported, 1~32. Default 8.
- `min_speed` — the slowest movement that counts as a moving target. Default 250.
- `speed_clip` — each frame's speed is capped at this before it is added up, so one spike cannot create a target. 0 = no cap. Default 500.
- `min_speed_sum` — the total the added-up speed must exceed over `confirm_frames` for the target to be reported. 0 = no check. Default 2000.
- `confirm_frames` (stationary) — consecutive frames a stationary target must be seen in before it is reported, 1~255. Default 3.

- `match_distance_cm` — how far a new sighting may be from where a stationary target was last seen and still count as the same target. Default 100.

Speeds are in the module's own unit. At the default sampling rate 500 is about 0.19 m/s; `kw307_set_sample_rate()` changes that scale.

`smoothing` is the same weight in thousandths as the range-gate IIR above, applied to the reported track instead of to the gate: 300 (0.3) moving, 60 (0.06) stationary by default.

> `kw307_set_motion_threshold()` is what a motion sensitivity level sets. Reading the level back after setting these four directly reports no level.

4.6 Line-of-sight filtering

A person blocks the radar. Someone standing directly behind them sends back almost nothing. So what the module sees at that angle is usually a ghost or clutter, not a second person.

That is why the module reports only the front target by default.

Turn it off to see every candidate:

```
kw307_en_los(false);   /* report everything, hidden or not */
    /* ... read every candidate ... */
kw307_en_los(true);    /* back to normal */
```

Turn line of sight off when two people in a queue must count as two, or to see every candidate the radar returns while you tune an installation.

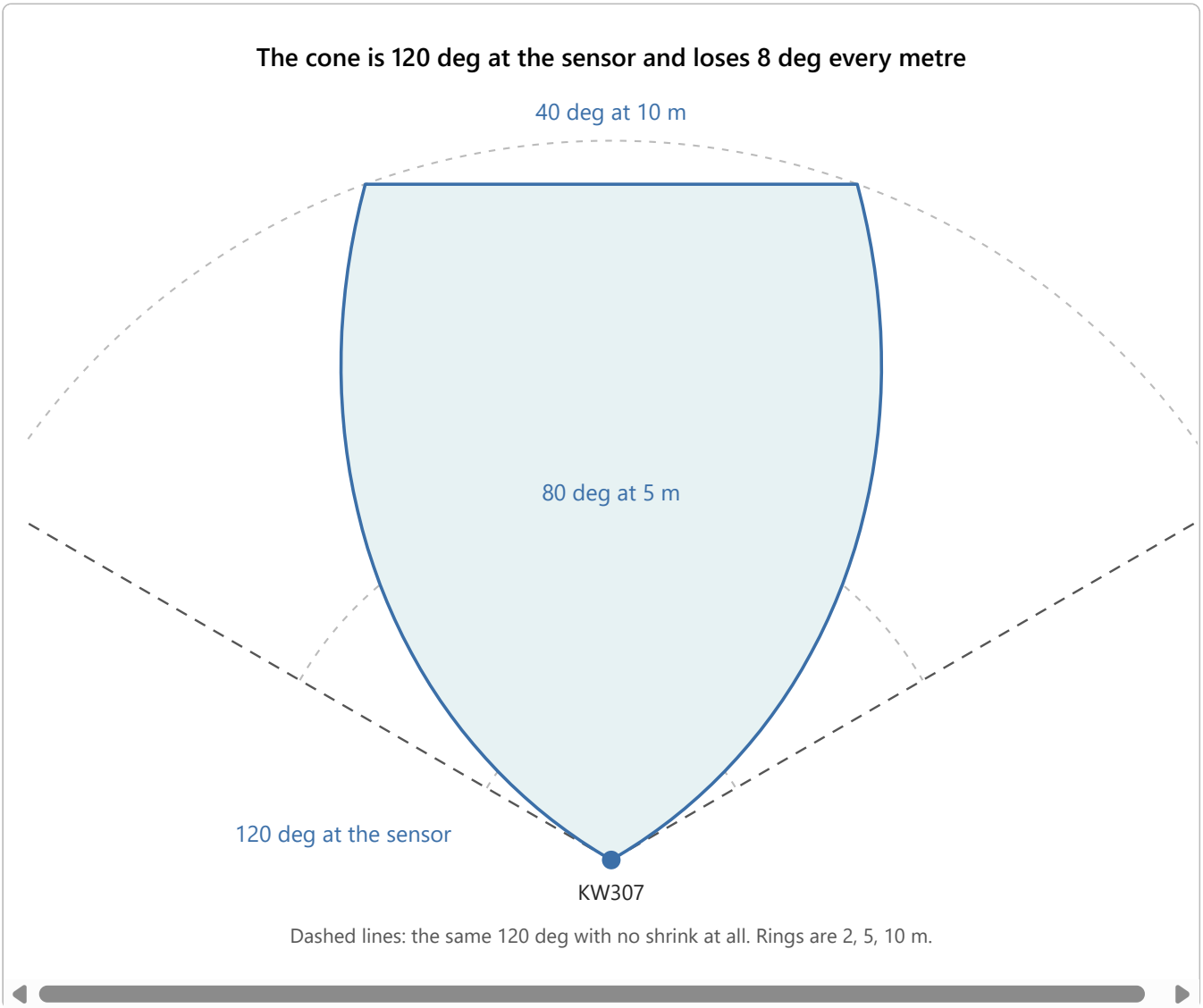
> **It also lets reflections through.** An echo behind a moving target is usually that target bounced off a wall. The module drops it for the same reason it drops a hidden person: it is behind something. With line of sight off, both come through. Expect more targets than there are people.

4.7 Detect range and field of view

In KW307, you can configure the detect range and the field of view. Objects outside this detection zone are ignored before tracking.

```
kw307_set_detect_range(min_cm, max_cm);          /* the room, in cm */
kw307_set_detect_fov(fov_deg, shrink_rate_deg);  /* the field of view, and how it narrows with distance */
```

A fixed FOV covers more floor the further out it reaches. Narrowing it with distance keeps the far end of the zone inside the room. The FOV never narrows below the limit set by `kw307_set_fov_min()`, 20° by default.



At the default 120° with a shrink of 8° per metre: 120° at the module, 80° at 5 m, 40° at 10 m.

4.8 Presence flag

The presence flag is a simple on/off indication. Any object that shows motion or micro-Doppler signatures inside the detection zone sets it. It sets as soon as a target is confirmed, and a hold time keeps it from flickering: after the last detection, the flag stays set for the hold time, in frames.

```
kw307_set_presence_hold_time(frames);   /* default 220 = 11 s */
```

Normal operation is 20 Hz, so 1 s = 20 frames; the most, 32767 frames, is about 1638 s (27 min). With adaptive power saving on (`kw307_en_update_rate_adas()`) the hold also runs in normal time. At a lower update rate with adaptive saving off, one frame is one report (at 1 Hz, about 1 s). `kw307_set_stationary_release_time()` counts the same way.

> **For a product's occupancy logic, we recommend using the tracker output or the range-gate data.** They report each target's magnitude, distance, angle and speed at a fixed rate, 20 Hz by default, so your logic can decide what counts as a person. The presence flag is set by any motion in the zone.

5 Application examples

Five worked configurations ship with the SDK, one per `DEMO_SCENE_*` in every `example_platform/*/main.c`. They are not presets: no call selects one. Each block is the same ordinary setters this guide describes, already tuned for a room.

SCENE	THE ROOM IT WAS TUNED FOR
`DEMO_SCENE_OUTDOOR`	A yard or a doorway, long and busy
`DEMO_SCENE_OFFICE`	A room someone works in, sitting still for long stretches
`DEMO_SCENE_DESK`	One person, close, a screen away
`DEMO_SCENE_URINAL`	A narrow bay, one occupant, quick turnover
`DEMO_SCENE_HAND_DRYER`	A hand held under the outlet, in a tight window

Pick the one whose room is closest to yours and read its block in `example_platform/*/main.c` — the baseband gain, the range, the field of view, the two sensitivity levels, the detection thresholds, the tracker counters and the holds, all in one place and all commented. Copy it and adjust from there.

`DEMO_SCENE_OFFICE` is the one the module already boots at: motion level 2, stationary level 1.

6 Function reference

Grouped by where each call acts on the diagram. Every setter has a matching `kw307_get_*` that reads the value back; those take the same pointers the setter takes values, and are omitted here.

6.1 Session

CALL	WHAT IT DOES
`kw307_init()` / `kw307_close()`	Open and close the link.
`kw307_update()`	Read the UART and call the registered callback. Call it on every pass of your main loop.
`kw307_radar_init()`	Re-initialise the module. Settings return to the last save; unsaved changes are discarded.
`kw307_save_settings()`	Write the current settings to flash. Wait about 1 s before the next call. Call it only after settings have changed, not on every pass of a loop: a call that saves a change rewrites the flash.
`kw307_factory_reset()`	Return every setting, and the output mode, to the default.
`kw307_clear_state()`	Drop stale targets without touching settings.
`kw307_check_fw_version()`	Check the module firmware is compatible with this SDK: `0` compatible, `1` firmware 2.7.6, negative on a link error.

6.2 Filters

CALL	WHAT IT DOES
`kw307_set_motion_sensitivity(level)`	Moving band as one level, 0~9, or -1 for any motion at all.
`kw307_set_stationary_sensitivity(level)`	Stationary band as one level, 0~9.
`kw307_set_moving_hpf_corner(corner, nd)`	Moving high-pass, directly.
`kw307_set_moving_lpf_corner(corner)`	Moving low-pass.
`kw307_set_stationary_hpf_corner(corner, nd)`	Stationary high-pass.
`kw307_set_stationary_lpf_corner(corner)`	Stationary low-pass.
`kw307_set_moving_background(update)`	Freeze or keep adapting, moving band. Not saved.
`kw307_set_stationary_background(update)`	Freeze or keep adapting, stationary band. Not saved.
`kw307_en_moving_env_subtraction(on)`	Subtract the environment, or pass it through. Not saved.
`kw307_en_stationary_env_subtraction(on)`	The same, stationary band. Not saved.
`kw307_en_stationary_hold(on)`	Freeze the stationary detections while you disturb that band. Not saved.
`kw307_en_analog_hpf(on)`	The analog high-pass, in the receiver. On by default.
`kw307_en_analog_hpf_swcap(on)`	Its switched-capacitor path. On by default; turn both off to see something that never moves.

6.3 Detection and reporting

CALL	WHAT IT DOES
`kw307_set_detect_range(min_cm, max_cm)`	Detect range: the near and far limits, in cm.
`kw307_set_detect_fov(fov_deg, shrink_rate_deg)`	Field of view, and how it narrows with distance.
`kw307_set_fov_min(fov_deg)`	The narrowest the shrink can make the FOV.
`kw307_set_mag_threshold(moving, stationary)`	The detection threshold per band.
`kw307_en_noise_floor_report(on)`	Report the room's noise floor in an empty slot's magnitude.
`kw307_set_presence_hold_time(frames)`	How long the presence flag holds after the last detection, in frames (20 = 1 s).
`kw307_en_gpio_presence_flag(on)`	Mirror the presence flag on the GPIO pin. Not saved.
`kw307_set_output(mode)`	Which frame the module sends: `_NORMAL`, `_NORMAL_WITH_SPD`, `_NORMAL_WITH_SPD_RG`, or `_OFF`. Not saved.
`kw307_on_frame(cb)`	Receive `KW307_OUTPUT_NORMAL` frames.
`kw307_on_frame_spd(cb)`	Receive `KW307_OUTPUT_NORMAL_WITH_SPD` frames.
`kw307_on_frame_spd_rg(cb)`	Receive `KW307_OUTPUT_NORMAL_WITH_SPD_RG` frames.

6.4 Custom frames

CALL	WHAT IT DOES
`kw307_out_clear(c)`	Every block off - the start of a new shape.
`kw307_out_normal_with_spd(c)` / `kw307_out_normal_with_spd_rg(c)`	Start from the `KW307_OUTPUT_NORMAL_WITH_SPD` or `KW307_OUTPUT_NORMAL_WITH_SPD_RG` shape instead.
`kw307_out_presence(c, on)`	The presence flag.
`kw307_out_moving_trk(c, on, full, speed)`	The two moving trackers, whole or state only, with or without speed.
`kw307_out_stationary(c, nearest, second)`	The nearest and second-nearest stationary targets.
`kw307_out_stationary_trk(c, on, full)`	The four stationary-band trackers.
`kw307_out_room_max(c, on)`	The room's peak magnitude per band.
`kw307_out_rg(c, start_rg, end_rg, moving, stationary, mag_only)`	Range gates, per band.
`kw307_out_len(c)`	The frame size that shape will produce.
`kw307_set_output_config(c)`	Send the shape. Nothing reaches the module before this.
`kw307_set_output_rg(start_rg, end_rg, en_moving_trk, en_stationary_trk)`	The board-tuning shape in one call: both bands' gates, trackers optional.
`kw307_on_frame_f3(cb)`	Receive any shape, decoded into `KW307FrameF3_t`.
`kw307_dec_frame_f3(data, len, out)`	Decode one from payload bytes, for your own transport.

6.5 Trackers

CALL	WHAT IT DOES
`kw307_en_los(on)`	Drop targets standing directly behind a nearer one. On by default; turn it off to see every candidate.
`kw307_set_moving_tracker_cnt(cnt_limit, display_th)`	Moving counters, in frames.
`kw307_set_stationary_tracker_cnt(cnt_limit, display_th)`	Stationary counters.
`kw307_set_motion_threshold(frames, min_speed, clip, sum)`	What counts as movement. A level sets these.
`kw307_set_moving_smoothing(smoothing)`	How steadily a moving target's position follows.
`kw307_set_stationary_tracker_opt(confirm, smoothing, match_cm)`	What it takes to report a stationary target, and how steadily.
`kw307_set_RangeGate_IIR(moving, stationary)`	How much each gate's magnitude, distance, angle and speed are averaged over frames.
`kw307_set_presence_hold_zone(on, distance_cm, fov_deg)`	Hold presence while somebody stands still in a near forward zone.
`kw307_set_stationary_release_time(enable, frames)`	Drop a stationary target after this many frames (20 = 1 s).
`kw307_set_tracker_state_advanced_mode(advanced)`	Choose the advanced reporting form, which adds the in-between states, or the simple one (Empty / Valid).
`kw307_set_advanced_clutter_rejection(en, spd_th, dev_th)`	Reject repetitive motion such as a fan.

6.6 Front end and power

CALL	WHAT IT DOES
`kw307_set_gain(index)`	Baseband gain in the analog receive chain, 0 to +21 dB.
`kw307_set_equalizer_gain(index)`	Equalizer gain in the analog receive chain, 0 to +12 dB.
`kw307_set_baseband_att(index)`	Baseband attenuator, 0 to -12 dB. Stationary levels 7~9 set it too.
`kw307_set_digital_gain(index)`	Digital gain.
`kw307_set_equalizer_order(order)`	Equalizer order.
`kw307_set_antenna(pa_num, rx_ant_num, with_angle)`	Which antennas are in use, and whether the angle is measured.
`kw307_set_sample_rate(sel)`	Sampling rate. The filter equations assume the 2 kHz default.
`kw307_set_update_rate(rate)`	Update rate, as one of three named settings. Not saved.
`kw307_en_update_rate_adas(on)`	Adaptive power saving, on top of the cycle. Not saved.
`kw307_set_update_cycle(sleep_frames, skip_frames, output_count, crystal_off)`	The update cycle itself, for a rate the three do not name. Not saved.
`kw307_set_power_mode(mode)`	Power mode. After DOWN, sensing resumes only on NORMAL. Not saved.

7 UART protocol

Everything above is the SDK speaking this protocol for you. This section is the protocol itself. You need the bytes in two cases: a host that does not run the SDK, and a link that misbehaves and needs a logic analyser on it.

7.1 UART link settings

Baud rate	115200
Data bits · parity · stop bits	8 · none · 1
Flow control	None
Logic level	3.3 V TTL — low 0 V, high 3.3 V
Byte order	Big-endian, MSB first, payloads included

7.2 UART frame format

```
00 00 00 00 00   AA  55   CMD   LEN_H LEN_L   DATA ...   CRC_H CRC_L
└── wake-up ──┘  └SOF┘
```

FIELD	
Wake-up	At least five `0x00` bytes before **every** frame you send. The module drops into an idle low-power state between commands, and these wake it; they are discarded. It has to be `0x00` — a run of them gives the RX line the repeated edges the wake detector is watching for, and another filler byte is not a substitute.
SOF	`AA 55` to write or to stream, `AA 56` to read. The second byte is the only thing that distinguishes a write from a read request.
CMD	What the frame is about. The same CMD number is used for the write and the read of a setting.
LEN	Payload length, big-endian, not counting CRC.
CRC	CRC-16, polynomial `0x8005`, seed `0xFFFF`, big-endian, computed over CMD + LEN + DATA. The SOF and the wake-up bytes are outside it.

> **Send the wake-up bytes and the frame as one stream.** A gap lets the module go back into that idle state before the command arrives. The command is then lost with no error on either side: the frame was never received, so there is nothing to NACK. This is a common cause when a host that does not use the SDK receives no reply.

7.3 Write and read requests

A **write** is `AA 55 [CMD] …` . The module replies with an ACK frame on CMD `0xAC` whose payload carries the CMD it is answering and a status byte:

STATUS	
`0x00`	Done.
`0x01`	The value was out of range.
`0x02`	The module does not know that CMD — usually firmware older than the command.
`0x03`	CRC mismatch; the frame arrived damaged.
`0x04`	The length field did not match the payload.

A **read** is `AA 56 [CMD]` with an empty or selector-only payload, and the module answers on the same CMD with the current value.

Output frames streamed by the module (`0xF0` , `0xF3`) arrive without a request, with the same header and CRC but no wake-up bytes, and no ACK: they are output, not a reply.

7.4 Output frame layouts

This section is for a host that is **not** running the SDK — one written in Python, C#, or on a PLC. If you are using the SDK, you never see these bytes: the decoders hand you a struct. `core/KW307_protocol.h` says what every field means, including the `state` values and which ones mean a Valid target. That header is the reference; this is only the byte layout, which no header carries.

Each of the three output modes has **one fixed layout**. Every multi-byte value is 16-bit **big-endian**, and distances on the wire are in **millimetres** — the SDK converts them to centimetres, a host parsing the bytes does not.

A **tracker** is the same 8 bytes everywhere it appears:

OFFSET	FIELD	TYPE
+0~1	`distance_mm`	uint16
+2~3	`angle_tenth`	int16, 0.1 degree
+4~5	`mag`	uint16
+6	`state`	uint8
+7	`counter`	uint8

`KW307_OUTPUT_NORMAL` , CMD `0xF0` , 27 bytes

OFFSET	FIELD
0~1	`frame_index`
2	`presence_flag`
3~10	moving target 1
11~18	moving target 2
19~26	stationary target

`KW307_OUTPUT_NORMAL_WITH_SPD` , CMD `0xF3` , 34 bytes

OFFSET	FIELD
0~1	`frame_index`
2~4	`B0 2C 09` — the shape bytes, always these three in this mode
5	`presence_flag`
6~13	moving target 1
14~21	moving target 2
22~25	the two moving targets' speeds, int16 each
26~33	stationary target

`KW307_OUTPUT_NORMAL_WITH_SPD_RG` , CMD `0xF3` , 66 bytes

OFFSET	FIELD
0~1	`frame_index`
2~4	`30 2E 09` — the shape bytes, always these three in this mode
5	`presence_flag`
6~13	moving target 1
14~21	moving target 2
22~25	the two moving targets' speeds, int16 each
26~33	stationary target
34~41	`distance_mm` , stationary-band gates 0, 1, 2, 3
42~49	`angle_tenth` , gates 0~3
50~57	`mag` , gates 0~3
58~65	speed, gates 0~3

Those are the raw per-gate readings, before grouping and before any tracker. A gate with nothing in it reads as noise, not as zero.

> **Tell the two `0xF3` modes apart by bytes 2~4, never by length.** The two differ in those three bytes. A length check cannot tell a truncated frame of one shape from a whole frame of the other. `KW307_OUTPUT_NORMAL` has no shape bytes — it is `0xF0`, and only ever 27 bytes.